

Loss Functions, Training Loop, Tensor Datasets & Dataloaders

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



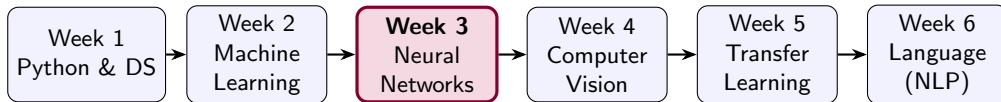
جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 3, Day 16, Lecture

4 October 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. Loss Functions
4. The Training Loop
5. Datasets and DataLoaders
6. Reading Loss Curves
7. Olympiad Practice
8. Limitations and What Comes Next
9. Summary
10. References



Today: Day 16 of 42

- ▶ **Yesterday:** a neuron, a network of them, the chain rule and `loss.backward()`, and the four-move loop in PyTorch, run on the whole table at every step.
- ▶ **Today:** the loop as it is really written: the **loss** that fits the task, **mini-batches** from a `DataLoader`, a **validation pass** every epoch, and how to **read the curve** it draws.
- ▶ **Tomorrow:** the dials of that loop: optimizers, learning-rate schedules, BatchNorm, dropout, early stopping.

Day 15's loop on MNIST: 20,000 images in one tensor, one gradient step per pass, the training loss as the only number.

```
for epoch in range(300):  
    loss = loss_fn(model(X_train), y_train)      # all 20,000 rows at once  
    opt.zero_grad(); loss.backward(); opt.step()  
    print(loss.item())                          # training loss only
```

What the loop decided	How
the loss	CrossEntropyLoss, taken on trust
how much data per step	everything, so one step per pass over the data
when to stop	after 300 steps, chosen before seeing anything
how good the model is on new rows	not measured; the curve was the training loss

It worked (0.957 on the test images) because MNIST is small and forgiving. The next tasks are not.

The problem

- ▶ **Memory.** MNIST as `float32` is $60,000 \times 784 \times 4$ bytes = 188 MB: fine. ImageNet is 1.28 million images of $224 \times 224 \times 3$: 193 GB. No table at once.
- ▶ **Steps.** Three passes over 20,000 images with one step per pass: test accuracy **0.595**. The same three passes in batches of 128 (157 steps each): **0.936**.
- ▶ **Blindness.** A 784-512-10 network on 1,000 images drives its training loss to 0.002 while the validation loss turns up after epoch 13. Day 15's loop prints the first number and never sees the second.
- ▶ **The loss on trust.** A price scored by MAE, a yes/no with a rare class: `CrossEntropyLoss` on everything costs points.

The pattern

The four moves were right. What is missing is everything around them: how the data arrives, which number is minimised, and which number decides when to stop.

In one sentence

A training loop is a contract between four pieces: a **Dataset** that says what one sample is, a **DataLoader** that serves shuffled mini-batches, a **loss** that fits the target and the judge's metric, and a **validation pass** every epoch whose curve tells you what to change and which epoch to keep.

- ▶ Mini-batches: many steps per pass, a little noise, memory that fits. The loop body stays Day 15's four moves.
- ▶ The loss: one per kind of target, applied to *logits*; weights for a rare class; L1Loss when the judge reads MAE.
- ▶ The curve: too big a rate, too small a rate, overfitting, underfitting, each with a shape and a fix.

Where you will meet it

Every notebook of Weeks 3 to 6 is this loop with a different model inside. On Friday the leaderboard scores the epoch you kept, not the last one you ran.

By the end of this session, you will be able to:

- ▶ **Choose** the PyTorch loss that fits a task (a number, a yes/no, one of k classes) and the judge's metric, and explain why the model outputs logits.
- ▶ **Compute** the cross-entropy of a prediction by hand and say what a loss of $\ln 10$ means.
- ▶ **Write** the full training loop: mini-batches, `train()` and `eval()`, a validation pass every epoch, the best epoch kept.
- ▶ **Build** a `TensorDataset` and a `DataLoader`, write your own `Dataset` class, and pick batch size and shuffling.
- ▶ **Read** a loss curve: too small or too big a learning rate, overfitting, underfitting, and what to change.
- ▶ **Move** a model and its batches to a GPU in three lines.

Training: **Loss Functions**

Definition

The **loss** is the number `backward()` differentiates: smooth (Day 09), fitted to the kind of target, and as close to the judge's metric as a smooth function can be.

Target	Last layer	Outputs	Loss (PyTorch)	Label dtype	Metric it serves
a number	1 neuron, no activation	$\hat{y}$	MSELoss, L1Loss	float	RMSE, MAE
yes / no	1 neuron, no activation	a logit z	BCEWithLogitsLoss	float 0/1	accuracy, F1, AUC
one of k	k neurons, no activation	k logits	CrossEntropyLoss	int64 class index	accuracy, macro F1

- ▶ “No activation” on the last layer is on purpose: the loss applies the sigmoid or the softmax itself (two frames on). The raw outputs are called **logits**.
- ▶ The label's dtype is part of the contract: `CrossEntropyLoss` wants int64 class numbers, the other two float32. The wrong dtype is this week's most common error.

For one sample, the softmax turns the k logits into probabilities; the loss is minus the log of the probability given to the *true* class:

$$\ell = -\log p_{\text{true}}, \quad \text{averaged over the batch.}$$

Three classes

Logits (2.0, 1.0, 0.5), softmax (0.629, 0.231, 0.140) (Day 15).

Truth is class 0: $\ell = -\log 0.629 = 0.464$. Truth is class 2: $\ell = -\log 0.140 = 1.966$.

Same logits, four times the loss: the loss punishes confidence in the wrong place, like Day 09's log loss.

- ▶ A network that knows nothing gives every class $1/k$: loss $-\log(1/k) = \ln k$; for 10 digits $\ln 10 = 2.303$, the number before training on Day 15.
- ▶ Yes/no is the same with $k = 2$: BCEWithLogitsLoss is $-\log p$ for a yes, $-\log(1 - p)$ for a no, $p = \sigma(z)$. It is not an accuracy: it keeps falling as the probabilities sharpen.

Why Logits: the Loss Applies the Sigmoid Itself

Take a yes/no neuron with logit $z = 40$ and truth “no”:

- ▶ $\sigma(40) = 1/(1 + e^{-40})$ rounds to exactly 1.0 in float32; the loss by hand, $-\log(1 - 1.0)$, is **infinite**, the gradient nan, the dials ruined.
- ▶ `BCEWithLogitsLoss(40, 0)` returns 40.0: it rewrites $-\log(1 - \sigma(z))$ as $\log(1 + e^z)$ and computes that safely.

The rule

The model outputs logits; the loss turns them into probabilities. Apply `sigmoid` or `softmax` yourself only to *read* a probability.

```
z = torch.tensor([40.0])
y = torch.tensor([0.0])
p = torch.sigmoid(z)           # 1.0
-torch.log(1 - p)              # inf
nn.BCEWithLogitsLoss()(z, y)   #
                                40.
```

```
probs = torch.softmax(logits, 1)
```

Same for k classes: `CrossEntropyLoss` contains the softmax; a softmax layer inside the model makes a double softmax, which trains slowly and wrong.

Day 09: on the admissions table 70.2% of students are rejected, and a model that says “no” to everyone scores 0.702. The loss has the same blind spot: the common class dominates the average.

```
w = torch.tensor([1.0, 2.36])          # a yes counts 2.36 times a no
loss_fn = nn.CrossEntropyLoss(weight=w)  # one weight per class
pw = torch.tensor([2.36])
loss_fn = nn.BCEWithLogitsLoss(pos_weight=pw)  # the weight of a yes
```

- ▶ The weight multiplies each sample's loss by the weight of its class: a missed “yes” now costs 2.36 misses of a “no”. The usual choice is 1/share of each class, scaled so the common class has weight 1.
- ▶ Use it when the metric cares about the rare class (recall, F1, macro F1, Day 09); leave it out when the metric is plain accuracy.
- ▶ It changes the loss, not the data: a cheaper alternative to copying the rare rows.

Training: **The Training Loop**

```
def fit(model, train_loader, X_val, y_val, lr, epochs):  
    opt = torch.optim.SGD(model.parameters(), lr=lr)  
    history = []  
    for epoch in range(epochs):  
        model.train()  
        for xb, yb in train_loader:                # one mini-batch at a time  
            loss = loss_fn(model(xb), yb)           # 1. predict    2. loss  
            opt.zero_grad(); loss.backward()         # 3. gradient  
            opt.step()                               # 4. update  
        model.eval()  
        with torch.no_grad():                        # validation pass: no  
            slopes  
            out = model(X_val)  
            history.append((loss_fn(out, y_val).item(),  
                            (out.argmax(1) == y_val).float().mean().item()))  
    return history
```

- Two rings around Day 15's four moves: the **batch loop** inside (64 rows at a time) and the **validation pass** after each epoch (loss and metric on rows never trained on). `history` is the curve you plot to pick the epoch.

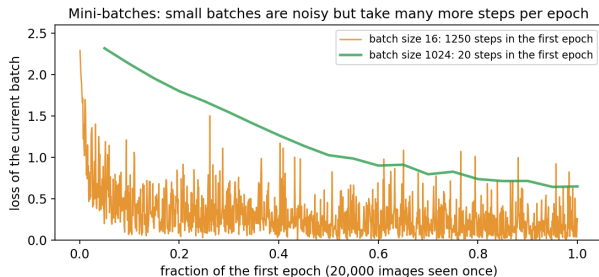
Why Mini-Batches: Many Steps per Pass

batch size	updates per epoch	seconds per epoch	test accuracy after 3 epochs
16	1,250	0.40	0.958
128	157	0.08	0.936
1,024	20	0.07	0.893
20,000 (the whole table)	1	0.07	0.595

784-128-10 network, 20,000 MNIST images, learning rate 0.1, `scripts/day16_training_figures.py`.

- ▶ An **epoch** is one pass over the training rows; in batches of 128 it is 157 small steps instead of one big one, for the same arithmetic.
- ▶ Small batches cost time (1,250 iterations) and a noisier slope; huge ones are fast per epoch but learn little. The usual range is 32 to 256; start at 64.
- ▶ The noise also helps: a slope measured on 64 random rows jumps around, which shakes the walk out of flat regions (Day 17 builds on this).

What a Batch Loss Looks Like



Source: the first epoch of the runs in the table, loss of each mini-batch as it is processed; `scripts/day16_training_figures.py`.

- ▶ Each point is the loss of one batch, so the curve of batch size 16 jumps: 16 rows are a rough estimate of the whole table. Both curves fall; the small-batch one falls earlier because it has taken more steps.
- ▶ Report the *mean* loss over an epoch (or the validation loss), never one batch's loss: a single batch can be easy or hard by chance.

```
model.train()          # training mode
for xb, yb in loader:
    ...                # the four moves

model.eval()           # prediction mode
with torch.no_grad():  # no recording
    out = model(X_val)
    probs = torch.softmax(out, dim=1)
```

- ▶ `model.train()` and `model.eval()` switch the layers that behave differently in training and in prediction. Today's layers do not care; tomorrow's dropout and BatchNorm do, and forgetting `eval()` then costs accuracy.
- ▶ `torch.no_grad()` stops autograd from recording: faster and half the memory, for any forward pass that is not followed by `backward()`.
- ▶ Both are habits: every validation pass, every prediction, every submission file is written inside them.

```
import copy
best_acc, best_state = 0.0, None
for epoch in range(epochs):
    ...                                     # train, then validate
    if val_acc > best_acc:                 # the judge's metric, not the loss
        best_acc = val_acc
        best_state = copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)         # the model you submit
```

The 1,000-image run

Validation loss is lowest at epoch 13 (0.392, accuracy 0.885); at epoch 120 it is 0.479 while accuracy is 0.891 and the training loss is 0.002. The loss rose because the wrong answers became *confident*, not more numerous.

- `state_dict()` holds every dial; copying it at the best epoch is the whole trick (Day 17's early stopping automates the stop). Keep the epoch best on the **metric the task is scored by**: accuracy here, the loss when the judge reads log loss or ROC-AUC.

```
device = "cpu"
if torch.cuda.is_available():
    device = "cuda"
model = model.to(device)
for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(
        device)
    ...
out = model(X_val.to(device))
preds = out.argmax(1).cpu()
```

- ▶ Model and batch must live on the same device; a CPU tensor meeting a GPU model raises an error that names both.
- ▶ On Colab: Runtime, Change runtime type, T4 GPU. Results come back to the CPU with `.cpu()` before NumPy or pandas see them.

network, one epoch	CPU	GPU
784-128-10 (101,770 dials)	0.10 s	0.35 s
784-2048-2048-10 (5.8 M dials)	2.0 s	0.8 s

Measured on the laptop that built this deck (CPU against its Apple GPU), batches of 64, 20,000 images.

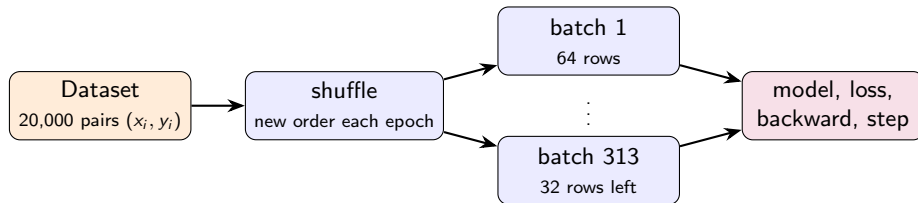
The rule

A GPU multiplies big matrices fast and pays a fixed cost per call. Tiny network, tiny batches: the CPU wins. Millions of dials, images, Week 4 onwards: the GPU wins by far.

Training: **Datasets and DataLoaders**

Definition

A **Dataset** is an object that behaves like a list of samples: `len(ds)` says how many, `ds[i]` returns one (x, y) pair. A **DataLoader** wraps a Dataset and hands out **mini-batches**: it shuffles the indices every epoch, cuts them into groups of `batch_size`, and stacks each group into two tensors.



- Day 04's objects: a Dataset is a class with `__len__` and `__getitem__` ("making your object behave like a list"). The DataLoader only needs those two methods.
- The samples never have to fit in memory at once: `ds[i]` can read one image from disk when asked.

```
from torch.utils.data import TensorDataset, DataLoader
train_ds = TensorDataset(X_train, y_train)    # tensors in memory
len(train_ds), train_ds[0]                  # 20000, (tensor of 784, tensor(5))

train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
val_ds = TensorDataset(X_val, y_val)
val_loader = DataLoader(val_ds, batch_size=1000)
len(train_loader)                          # 313 batches: 312 of 64, then one of 32
xb, yb = next(iter(train_loader))
xb.shape, yb.shape                        # torch.Size([64, 784]), torch.Size([64])
```

- ▶ $20,000/64 = 312.5$: the loader gives 312 full batches and one of 32. `drop_last=True` throws the small one away; usually keep it.
- ▶ `shuffle=True` for training only; a fixed validation order keeps the scores comparable.
- ▶ One for `xb, yb` in `train_loader` is one **epoch**; its body is Day 15's four moves on 64 rows.

```
from torch.utils.data import Dataset
from PIL import Image

class DigitFiles(Dataset):
    def __init__(self, paths, labels):
        self.paths, self.labels = paths, labels
    def __len__(self):
        return len(self.paths)
    def __getitem__(self, i):
        img = Image.open(self.paths[i])
        # read one file
        x = torch.tensor(np.array(img) / 255.0,
                        dtype=torch.float32).
                        reshape(-1)
        return x, self.labels[i]

ds = DigitFiles(paths, labels)
loader = DataLoader(ds, batch_size=64,
                    shuffle=True)
```

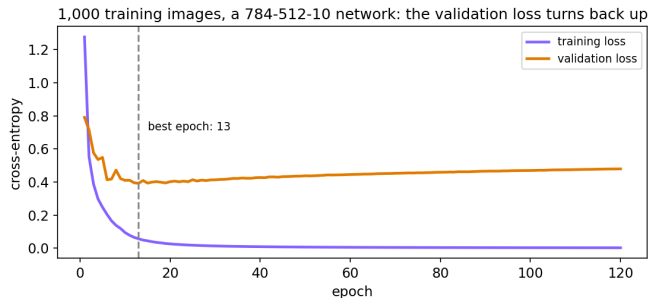
- ▶ `__init__` stores what is cheap (paths, labels); `__getitem__` does the work for one sample, when asked: 100,000 images on disk train in 1 GB of RAM, one batch in memory at a time.
- ▶ Anything can go into `__getitem__`: a crop, a flip, a normalisation (Week 4's augmentations); `num_workers=4` reads files in parallel while the GPU computes.

Dial	Training	Validation and test
batch_size	32 to 256; smaller is noisier, bigger needs more memory	as large as memory allows
shuffle	True	False
drop_last	False, unless a layer needs equal batches (Day 17)	False
num_workers	2 to 4 when samples are read from files	same

```
from torchvision import datasets, transforms
mnist = datasets.MNIST(root="data", train=True, download=True,
                       transform=transforms.ToTensor())
len(mnist), mnist[0][1]          # 60000, 5
mnist[0][0].shape                 # torch.Size([1, 28, 28])
loader = DataLoader(mnist, batch_size=64, shuffle=True)
```

torchvision ships MNIST, CIFAR-10 and more as Datasets with `__getitem__` already written; transform is the function applied to each image on the way out. Week 4 starts here.

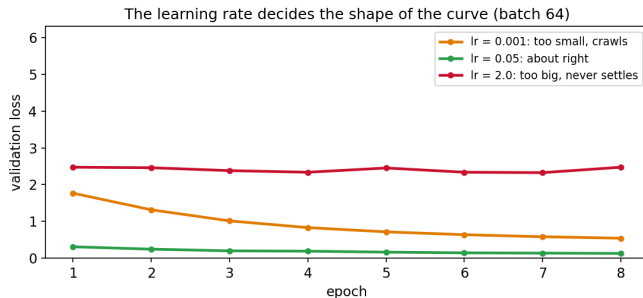
Training: **Reading Loss Curves**



Source: 1,000 MNIST training images, 784-512-10 network, batches of 32, learning rate 0.05, validation on 2,000 held-out images; `scripts/day16-training_figures.py`.

- ▶ Training loss only falls (to 0.002: the 1,000 images are memorised). Validation loss falls, turns at epoch 13, then climbs: Day 09's alarm, drawn by a network.
- ▶ Today's answer is the best epoch. Tomorrow's are dropout, weight decay and early stopping; the lasting one is more data.

The Learning Rate Draws the Shape



Source: 784-128-10 network, 20,000 MNIST images, batches of 64, eight epochs for each learning rate; scripts/day16_training_figures.py.

- ▶ 0.001 crawls: still 0.55 after eight epochs (accuracy 0.868). 0.05 reaches 0.13 (0.962). 2.0 never leaves $\ln 10 = 2.3$ (0.105): every step jumps across the valley.
- ▶ A loss stuck at $\ln k$ therefore has two readings, too small or too big. Try the rate $\times 10$ and $\nabla \cdot 10$ for one epoch each before anything else.

What the curve shows	Most likely cause	What to change
loss is nan or grows	learning rate too big	divide it by 10 (Day 08)
loss stuck at $\ln k$ from the start	rate too big or too small; inputs not scaled; labels wrong dtype	rate $\times 10$ and $\nabla \cdot 10$; scale the inputs; <code>int64</code> labels
falls, but slowly	rate too small; batches too big	rate $\times 10$; smaller batches
training falls, validation turns up	overfitting	keep the best epoch; dropout, weight decay (Day 17); more data
both stay high	underfitting	more units or layers; more epochs; better columns
validation jumps up and down	batches too small; validation set too small	bigger batches, lower rate; more validation rows
validation better than training	<code>eval()</code> forgotten, or a leak (Day 09)	call <code>eval()</code> ; check the split

Read the curve before touching the model: the first four rows cost one line of code to fix, the others cost an afternoon.

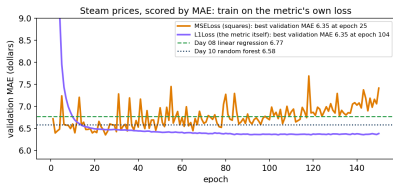
Training: **Olympiad Practice**

Task (Day 08, revisited)

3,000 games with Day 08's columns (Metacritic score, review counts, owners, release year, genres), target Price in dollars, scored by **MAE** on 600 hidden games: full points below 6.6, 50 points below 7.0, 40 below 8. Day 08's line reached 6.77, Day 10's forest 6.58. Train a 23-64-1 network with the loop of today.

- ▶ **Question 1:** the judge reads MAE. Which loss do you give backward(), and what goes wrong with the other one?
- ▶ **Question 2:** the loop runs 150 epochs. Which epoch's dials do you submit, and which number decides?
- ▶ **Question 3:** the review counts run from 0 to a million and two genre columns are constant. What happens to the loop if you standardise them as they are?

Adapted from the ONIA (Romanian AI Olympiad) official problem-proposal example "Predicting Game Prices on the Steam Platform", Olimpiada-AI, 2024/25; data from the Kaggle Steam Games data set.



Source: Day 08's features, counts logged, 2,400 / 600 split, batches of 64, rate 0.01;
scripts/day16-training-figures.py.

```
from copy import deepcopy
loss_fn = nn.L1Loss()      # the metric
    itself
...                          # the loop of
    today
with torch.no_grad():
    pred = model(Xv).squeeze(1)
    mae = (pred - yv).abs().mean()
if mae < best:
    best = mae
    state = deepcopy(model.state_dict())
```

1. L1Loss is the metric: best validation MAE 6.35 at epoch 104, still 6.38 at 150. MSELoss also touches 6.35 (epoch 25), then chases the expensive games up to 7.41: squares fear outliers (Day 09).
2. Submit the dials of the best validation-MAE epoch, not the last: with MSE that is full marks against 40 points.
3. Log the counts (Day 10) and leave a constant column alone: a spread of 0 sends a feature to 10^6 and the loss to the thousands. Both nets beat the forest (6.58) and the line (6.77).

Training: **Limitations & What Comes Next**

- ▶ **Plain gradient descent is slow and touchy.** The learning rate that works for one network fails for the next, and a fixed rate cannot be large at the start and small at the end. Day 17 adds momentum, Adam and schedules.
- ▶ **Stopping at the best epoch is a patch, not a cure.** The validation loss turned back up because the network memorised 1,000 images; dropout, weight decay and more data attack the cause (Day 17).
- ▶ **The validation score is spent a little every time you look at it.** Choosing the epoch, batch size and loss on it is tuning (Days 09 and 13); the test set stays untouched.
- ▶ **The loss is not the metric.** Cross-entropy keeps falling after accuracy stops moving, and L1Loss is not quite MAE on the hidden rows. Track the judge's metric on validation rows as well as the loss.

The common mistake

Reporting the *training* loss of the last epoch as the result. The last epoch is often the most overfitted one, and the training loss says nothing about new rows. Report the validation metric of the epoch you kept.

- ▶ **Tomorrow (Day 17):** optimizers (momentum, Adam), learning-rate schedules, BatchNorm, dropout and early stopping: the dials that make the loop of today converge faster and generalise better.
- ▶ **Days 18 to 20, self-study:** activation functions in depth, tensors and DataLoaders in depth, then the Olympiad habits: reading curves, choosing dials, submitting the right epoch.
- ▶ **Week 4:** the same loop and the same DataLoader, with convolutional layers reading the images unflattened.

Training: **Summary**

- ▶ **One loss per task:** `MSELoss` or `L1Loss` for a number, `BCEWithLogitsLoss` for yes/no, `CrossEntropyLoss` for k classes; the model outputs logits and the loss applies the sigmoid or softmax safely.
- ▶ **Cross-entropy** is $-\log p_{\text{true}}$: In k before learning, lower as the right class gains probability; class weights make a rare class count.
- ▶ **The loop:** `train()`, the four moves per mini-batch, `eval()` and `no_grad()` for a validation pass every epoch, and the best epoch kept.
- ▶ **Mini-batches:** many updates per pass over the data, a little noise, memory that fits; `Dataset` says what one sample is, `DataLoader` shuffles and batches it.
- ▶ **Reading curves:** nan or growth means the rate is too big, a crawl means too small, training down with validation up means overfitting, both high means underfitting.
- ▶ **GPU:** `device`, `model.to(device)`, `xb.to(device)`; it pays when the matrices are big.

Training: **References & Resources**

- [1] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS* 32. Documentation of `torch.nn` losses, `torch.utils.data` and `torchvision.datasets`.
<https://pytorch.org/docs>
- [2] LeCun, Y., Cortes, C. and Burges, C. *The MNIST database of handwritten digits*, loaded through OpenML (`mnist_784`). The Day 15 split (20,000 training, 10,000 test images) is reused for every curve of this deck.
- [3] ONIA (Romanian AI Olympiad), official problem-proposal example *Predicting Game Prices on the Steam Platform*, Olimpiada-AI, 2024/25; data from the Kaggle Steam Games data set. Day 08's practice task, revisited with a network.
- [4] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, chapters 6 and 8: cost functions, output units, mini-batch stochastic gradient descent. <https://www.deeplearningbook.org>
- [5] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapters 10 and 11: training loops, learning-rate behaviour, reading learning curves.
- [6] All plots: `slides-builder/scripts/day16_training_figures.py` (PyTorch runs on real data; it also prints every number quoted in this deck).